

UNIDAD III. DATOS SEMIESTRUCTURADOS

En el contexto de los sistemas de información y la gestión de datos, los datos semiestructurados constituyen una categoría intermedia entre los datos estructurados que están típicamente organizados en tablas relacionales y los datos no estructurados como imágenes, textos libres o videos.

Estos datos se caracterizan por no ajustarse completamente a un esquema rígido, pero aun así contienen información estructural que permite su interpretación y análisis.

Los modelos semiestructurados no siguen un modelo tabular fijo; en cambio, presentan una estructura jerárquica, generalmente representada mediante etiquetas o marcadores que describen las relaciones entre los datos. Estos datos se almacenan frecuentemente en formatos como XML (Extensible Markup Language), JSON (JavaScript Object Notation) o YAML¹ (Yet Another Markup Language), donde la organización de la información se expresa mediante una sintaxis que combina contenido y metadatos.

Una de las características distintivas de los datos semiestructurados es la flexibilidad del esquema, donde los diferentes elementos pueden tener distintas estructuras o atributos, y el esquema puede evolucionar sin necesidad de redefinir completamente la base de datos. Esta propiedad es especialmente útil en entornos donde los requisitos cambian con rapidez, como en aplicaciones web, procesamiento de documentos y almacenamiento de información heterogénea.

IMPORTANTE

En este material veremos sintéticamente un desarrollo conceptual, la profundización se encuentra en el material y bibliografía de la cátedra.

¹ Actualmente se lo entiende como: Ain't Markup Language, o YAML no es un lenguaje de marcado.

BASE DE DATOS II**CONTENIDOS Y ACTIVIDADES****LOS ORIGENES**

Como los modelos de datos semiestructurados se orientan al uso de múltiples tipos de datos, muy diferentes de los utilizados en las tradicionales Bases de Datos del modelo relacional, puede pensarse que son mucho más modernos, pero el SGML (Standard Generalized Markup Language) surge en la década de 1980 como una necesidad de estandarizar la representación de documentos electrónicos complejos, con el objetivo de lograr independencia entre el contenido, la estructura lógica y la presentación visual de los documentos.

ESCRIBIENDO EN LENGUAJE DE MARCADO GENERAL

El Standard Generalized Markup Language (SGML) es un estándar internacional (ISO 8879) para la definición de lenguajes de marcado que permiten describir la estructura y el contenido de documentos de forma independiente de su presentación. SGML no es un lenguaje de marcado por sí mismo, sino un marco para definir lenguajes de marcado personalizados, como lo son HTML y XML, que derivan directamente de este estándar.

La creación de un documento con estructuras SGML requiere primeramente la Definición de Tipo de Documento (DTD - Document Type Definition) que especifica el conjunto de reglas que definen la estructura válida de un documento SGML y luego que el documento propiamente dicho se ajuste estrictamente a su DTD.

En la definición del tipo de documento se describen los elementos permitidos y sus relaciones jerárquicas, se detallan los atributos de los elementos, el orden y la cantidad de apariciones de los elementos hijos y las entidades, que funcionan como macros² o referencias a texto reutilizable.

Una DTD que definirá el documento con estructura SGML se vería de la siguiente manera:

<!ELEMENT libro (titulo, autor, capitulo)>

<!ELEMENT titulo (#PCDATA)>

<!ELEMENT autor (#PCDATA)>

² Es decir, las entidades en SGML funcionan como referencias reutilizables de texto que permiten simplificar y estandarizar contenido repetido en un documento, similares a lo que realizan las macros en los lenguajes de programación.

BASE DE DATOS II**CONTENIDOS Y ACTIVIDADES**

<!ELEMENT capitulo (titulo, seccion)>

<!ELEMENT seccion (titulo, #PCDATA)>

Con la DTD, se construye el documento con estructura SGML que debe ajustarse a la estructura definida en la DTD, lo cual daría como resultado siguiendo la DTD anterior:

<libro>

<titulo>Introducción a la Programación</titulo>

<autor>María Gómez</autor>

<capitulo>

<titulo>Variables y Tipos de Datos</titulo>

<seccion>

<titulo>Enteros y Reales</titulo>

Los tipos numéricos básicos en programación...

</seccion>

</capitulo>

</libro>

PRO Y CONTRA

Los modelos semiestructurados son muy buenos para adaptarse a datos complejos y jerárquicos, poseen una mayor capacidad para integrar fuentes de datos diversas y al representarse mediante formatos como XML o JSON, que emplean etiquetas o claves con nombres significativos, lo cual facilita que tanto una persona como un programa comprendan rápidamente el significado del dato sin necesidad de documentación adicional.

Por ejemplo, si pretendemos registrar que Carla tiene 22 años, en lenguaje de marcado escribiríamos dentro de una etiqueta el nombre como: <nombre>Carla</nombre> y la edad como: <edad>22</edad>, lo cual a simple vista nosotros interpretaríamos fácilmente que el dato "22" corresponde a la edad de alguien llamada Carla, del mismo modo resultaría sencillo de identificar estos datos para un sistema informático.

BASE DE DATOS II**CONTENIDOS Y ACTIVIDADES**

Los datos semiestructurados también presentan desafíos, como la complejidad en la consulta y validación, y un menor rendimiento en operaciones transaccionales comparado con modelos relacionales.

Esto sucede porque los modelos semiestructurados al no tener un esquema fijo y uniforme a diferencia de las bases de datos relacionales, donde cada tabla tiene un esquema rígido con columnas y tipos de datos bien definidos, en los datos semiestructurados los elementos pueden tener estructuras diferentes dentro de un mismo conjunto de datos o pueden haber elementos a los cuales le falten atributos o encontrarse dentro del registro con datos registrados de forma ambigua o variable para un mismo valor por ejemplo, una fecha puede estar como texto o número al mismo tiempo, todo esto dificulta la escritura de consultas precisas, ya que se deben prever múltiples casos y excepciones.

No menos importante es saber que operar con modelos desestructurados requiere que comprendamos la sintaxis de lenguajes como XPath, JSONPath o XQuery, que son más complejos que el clásico SQL.

En los sistemas tradicionales, la validación está dada por el cumplimiento de restricciones del esquema relacional, en cambio, en los datos semiestructurados el esquema puede ser opcional o flexible, por lo que no siempre hay una DTD (Document Type Definition o Definición de Tipo de Documento) en SGML/XML o un esquema JSON definido.

Validar que los datos cumplen con una estructura esperada implica procesar reglas complejas y múltiples niveles de anidamiento y puede requerirse el uso de herramientas especializadas para realizar validaciones completas, como validadores de DTD, XML Schema o JSON Schema.

A nivel técnico, a diferencia de los SGBD (Sistemas de Gestión de Bases de Datos) relacionales que poseen reglas implícitas de optimización avanzada para manejar índices compuestos, planes de ejecución optimizados, etc, los Sistemas de Gestión de Bases de Datos semiestructurados no siempre los tienen, esto repercute en el rendimiento de las consultas, especialmente cuando se trata de grandes volúmenes de datos con estructuras variables.

BASE DE DATOS II**CONTENIDOS Y ACTIVIDADES****PRINCIPIOS FUNDAMENTALES DE SGML**

Los SGML posibilitan la separación de contenido su presentación, la definición explícita de estructuras documentales, son extensibles y adaptables, resultan altamente compatibles para múltiples plataformas y aplicaciones, orientados a la web por el uso del lenguaje de marcas como el HTML.

El uso de SGML permite que los documentos se definan en términos de su significado y estructura, sin necesidad de incluir información sobre cómo deben ser presentados. Esto facilita la reutilización del contenido en diferentes formatos (impreso, web, móvil, etc.).

A través de las DTD, SGML proporciona una forma de describir formalmente la estructura válida de un conjunto de documentos. Esto es crucial en entornos donde se manejan grandes volúmenes de documentos técnicos o normativos, a esta característica se la denomina definición explícita de estructuras documentales.

SGML permite crear lenguajes de marcado adaptados a necesidades específicas. Por ello, ha sido la base para el desarrollo de lenguajes como HTML (HyperText Markup Language) diseñado para la web o XML (Extensible Markup Language) una versión simplificada de SGML, orientada a la interoperabilidad y la web semántica.

Definir un estándar común, SGML posibilita el intercambio seguro de información estructurada entre organizaciones, independientemente del software o hardware utilizado.

BASE DE DATOS II**CONTENIDOS Y ACTIVIDADES****VALIDACIÓN DE DOCUMENTOS SGML**

Como los documentos SGML se basan en la DTD, para saber si cumplen o no con las reglas definidas en su DTD, debe ser sometido a un proceso de validación, este proceso es esencial para asegurar la coherencia estructural del documento, permitir su interpretación automatizada y evitar ambigüedades.

Para realizar la validación, se utilizan procesadores SGML que analizan el documento y su DTD, generando errores en caso de discrepancias, como elementos no definidos, secuencias no válidas, o atributos mal utilizados.

La validación SGML permite garantizar la calidad y consistencia de la información, facilitando la interoperabilidad entre sistemas que utilizan estructuras documentales comunes y lograr la automatizar la transformación, búsqueda y presentación de documentos.

APLICACIONES TÍPICAS DE SGML

El Lenguaje de Marcado Generalizado Estándar (SGML) ha sido ampliamente utilizado en sectores como el de publicaciones científicas y técnicas, de documentación normativa y legal, en la industria aeroespacial y militar y para la gestión de contenidos empresariales.

Aunque SGML ha sido en gran parte reemplazado por XML en muchos entornos, su influencia permanece vigente, especialmente en sistemas heredados y entornos donde se requiere un control exhaustivo sobre la estructura documental.

BASE DE DATOS II**CONTENIDOS Y ACTIVIDADES****CREANDO ARCHIVOS SGML**

Comenzamos con la práctica, para lograr el resultado esperado te sugiero que vayas paso a paso, entender la secuencia lógica te permitirá entender el funcionamiento

HERRAMIENTAS NECESARIAS

No vamos a necesitar software complejo para iniciar la actividad, ya que toda la práctica requiere escribir texto, para eso tenemos programas sencillos pero potentes a disposición.

Recurso	Función principal	Recomendado para
Bloc de notas / Gedit	Edición simple de texto	Todos
Notepad++ / VSCode	Mejor legibilidad y estructura	Nivel medio
Navegador web	Validación rápida de XML	Revisión básica
Validador XML online: xmlvalidation.com/	Validación formal con DTD	Comprobación

BASE DE DATOS II

CONTENIDOS Y ACTIVIDADES

¿MANOS A LA OBRA?

CREAR EL ARCHIVO DTD

PASO 1: ABRIR EL EDITOR DE TEXTO

El que prefieras, puede ser el Bloc de notas, Notepad++ o VSCode.

PASO 2: ESCRIBIR EL CONTENIDO DEL ARCHIVO DTD

Escribí el siguiente DTD el cual define una lista de libros dentro de un elemento raíz <ficha>, donde cada <libro> tiene título, autor, editorial, año y (opcional) ISBN y a su vez cada libro puede tener un atributo opcional "idioma".

```
<!ELEMENT ficha (libro+)>
```

```
<!ELEMENT libro (titulo, autor, editorial, año, isbn?)>
```

```
<!ELEMENT titulo (#PCDATA)>
```

```
<!ELEMENT autor (#PCDATA)>
```

```
<!ELEMENT editorial (#PCDATA)>
```

```
<!ELEMENT año (#PCDATA)>
```

```
<!ELEMENT isbn (#PCDATA)>
```

```
<!ATTLIST libro idioma CDATA #IMPLIED>
```

PASO 3: GUARDAR EL ARCHIVO

Para que sea más sencillo identificarlo luego, guarda el archivo con el nombre **fichas** y la extensión **.dtd**, seleccionando como tipo de archivo: Todos los archivos, codificado en UTF-8 si está disponible.

Si utilizas el bloc de notas, la opción de guardado se ve como en la figura:



¡COMPLETAMOS LA PRIMERA PARTE!

Ahora avanzaremos con la **SEGUNDA PARTE**, la creación del documento SGML/XML, para ello...

BASE DE DATOS II

CONTENIDOS Y ACTIVIDADES

ATENCIÓN

En esta práctica comenzaremos a ver una nueva forma de escritura para el registro de datos, se trata de utilizar un lenguaje con marcas.

PASO 1: ABRIR UN NUEVO DOCUMENTO EN EL EDITOR

Crearemos un nuevo documento que incluye dos libros, uno con ISBN y otro sin él (opcional).

PASO 2: ESCRIBIR EL CONTENIDO XML (COMPATIBLE CON EL DTD)

```
<!DOCTYPE ficha SYSTEM "fichas.dtd">
```

```
<ficha>
```

```
<libro idioma="Español">
```

```
<titulo>El nombre del viento</titulo>
```

```
<autor>Patrick Rothfuss</autor>
```

```
<editorial>Plaza & Janés</editorial>
```

```
<año>2007</año>
```

```
<isbn>9788401352836</isbn>
```

```
</libro>
```

```
<libro idioma="Inglés">
```

```
<titulo>Clean Code</titulo>
```

```
<autor>Robert C. Martin</autor>
```

```
<editorial>Prentice Hall</editorial>
```

```
<año>2008</año>
```

```
</libro>
```

```
</ficha>
```

PASO 3: GUARDAR EL ARCHIVO

En la misma carpeta donde se guardó el archivo **fichas.dtd**, guarda este nuevo archivo como **fichas.xml**.

BASE DE DATOS II

CONTENIDOS Y ACTIVIDADES

COMPROBANDO LA TEORÍA

Estamos en condiciones de realizar una comprobación de nuestros archivos creados para ver si los realizamos correctamente y cumple con la norma internacional, para ello tenemos dos formas, (hay muchas más), una en línea desde la herramienta oficial para validar documentos y otra fuera de línea directamente desde cualquier navegador web.

PRIMERA OPCIÓN: con navegador web

Abrir el archivo **fichas.xml** con algún navegador web, si nuestro documento tiene errores, se mostrará un mensaje de advertencia, si todo es correcto, se visualizará el contenido como texto estructurado.

SEGUNDA OPCIÓN: con un validador online como <https://www.xmlvalidation.com>

Ingresar en el enlace, luego copiar y pegar el contenido del archivo fichas.xml en el área de validación, activar la opción “Use external DTD” si está disponible (Puede variar la presentación del formulario en pantalla), subir también el archivo fichas.dtd si se ofrece esa posibilidad y finalmente hacer clic en Validate XML.

Si todo está bien, aparecerá un mensaje como “The document is valid”, pero...

¿Qué resultado nos informan los validadores con nuestro ejemplo? Creo que es momento de investigar :)

BASE DE DATOS II

CONTENIDOS Y ACTIVIDADES

PRACTICA

A continuación, realizaremos práctica con una serie de ejercicios, la actividad se divide en dos tramos, el primero consiste en realizar en grupo el TP asignado por el docente, el resto en forma individual, finalizado se debe presentar en la Plataforma en el **BUZÓN DE ENTREGA**:

- ☺ **fichas.dtd** (el archivo con la estructura).
- ☺ **fichas.xml** (el archivo con los datos).
- ☺ **Captura de pantalla de todo el proceso incluyendo el mensaje de validación en los dos formatos on line y off line.**
- ☺ **Una reflexión breve sobre cómo se relacionan los elementos del DTD con el documento SGML/XML.**

ACTIVIDAD GRUPAL/INDIVIDUAL

Cada grupo deberá crear el archivo DTD y su archivo XML con hasta 10 registro, luego realizar la validación en las dos formas vistas en clase y presentar sus resultados.

TP 1: Fichas bibliográficas.

Crear un DTD y un XML que represente fichas bibliográficas de libros. Incluir datos como título, autor, editorial, año, ISBN, y opcionalmente idioma. Validar el archivo.

TP 2: Registro de estudiantes.

Diseñar un DTD y XML que almacene información de estudiantes de una institución: nombre completo, DNI, carrera, año de inscripción, materias aprobadas (lista), y promedio general.

TP 3: Agenda de contactos.

Crear una estructura para una agenda de contactos personales con nombre, teléfono, email, dirección, cumpleaños, y etiqueta (familiar, trabajo, amigos).

TP 4: Recetario de cocina.

Desarrollar un DTD y XML que contenga una colección de recetas con nombre, ingredientes (lista), pasos de preparación, tiempo estimado y categoría (postre, principal, entrada).

TP 5: Catálogo de películas.

Modelar un XML con su DTD para representar un catálogo de películas: título, director, año, duración, género, elenco (lista de actores) y sinopsis.